

COMICCRAFT

AI Comic Story Creator using Gemini Models

PHASE-WISE PROJECT DOCUMENT

This document presents the project workflow phase by phase, following the milestones and activities described in the supplied ComicCraft project document.

Phase	Milestone	Main Focus
Phase 1	Model Selection & Architecture	AI model research, system architecture, development environment
Phase 2	Core Functionalities Development	Comic generation functions and FastAPI input processing
Phase 3	routes.py Development	Main application routes and AI workflow integration
Phase 4	Frontend Development	User interface and Jinja2 dynamic templates
Phase 5	Deployment	Local deployment, testing and verification

PHASE 1 — MILESTONE 1: MODEL SELECTION AND ARCHITECTURE

Activity 1.1 — Research and Select the Appropriate Generative AI Model

- The project requirements focus on three types of generated content: structured panel-by-panel comic outlines, full comic narration and character dialogues, and visual comic-style illustrations.
- The document describes evaluation of Google Gemini models and Hugging Face Diffusers / Stable Diffusion. The comparison considers creative-writing quality, prompt adherence and scene detail, image-generation speed and resolution, model stability, documentation and community support.
- The document records the selected combination as Gemini Flash for fast structured panel outlines, Gemini Pro for detailed narration and dialogue, and Stable Diffusion for comic-style illustrations.

Activity 1.2 — Define the Architecture of the Application

- The architecture is divided into three primary components: Frontend (HTML, CSS, Jinja2), Backend (FastAPI), and AI Integration (Google Gemini APIs and Hugging Face Diffusers).
- The frontend collects the story prompt, character name, setting, story tone and art style, then sends the input to the FastAPI backend.
- The backend receives input, coordinates AI generation, organizes the comic layout and exports the final comic as a PDF.

Activity 1.3 — Set Up the Development Environment

- Python and pip are installed, and a project-specific virtual environment is created.
- The supplied document lists FastAPI, Uvicorn, Jinja2, python-multipart, Google Generative AI, Diffusers, Transformers, FPDF, Pillow and accelerated among the required libraries.
- The server is run with `uvicorn app.main:app --reload`, with the application available at `http://127.0.0.1:8000` and FastAPI documentation at `/docs`.

PHASE 2 — MILESTONE 2: CORE FUNCTIONALITIES DEVELOPMENT

Activity 2.1 — Develop the Core Functionalities

- Generate Structured Comic Panel Outline — `generate_outline()` in `gemini_flash.py` generates a structured five-panel outline. Each panel contains a panel number, title, scene description and image-generation prompt.
- Generate Comic Story Narration and Dialogue — `generate_story()` in `gemini_pro.py` expands the panel outlines into comic-style narration and character dialogue.
- Generate Comic Illustrations — `generate_image()` in `image_generator.py` uses Stable Diffusion to create comic-style images from the generated image prompts and saves them under `static/panels`.
- Organize Comic Panels — `build_comic_layout()` in `layout_builder.py` matches generated images with panel stories and creates a structured layout.
- Export Comic to PDF — `save_pdf()` in `exporters.py` compiles the panels, images and narration into a multi-page PDF saved under `static/exports`.

Activity 2.2 — Implement the FastAPI Backend to Manage Routing and User Input Processing

- Routing logic is handled in `routes.py`.
- The HTML form collects story prompt, main character name, setting, tone and art style. FastAPI captures form inputs with `Form(...)`, while `PromptRequest` validates JSON input.
- The `/generate` workflow calls `generate_outline()`, `generate_story()`, `generate_image()` for each panel, `build_comic_layout()` and `save_pdf()`.
- The `/generate-comic/json` workflow provides similar processing through a JSON API request and returns layout data and the PDF path.
- The `/test-image` route directly tests image generation. AI responses are processed and passed to the Jinja2 templates.

PHASE 3 — MILESTONE 3: ROUTES.PY DEVELOPMENT

Activity 3.1 — Writing the Main Application Logic in routes.py

- The / route loads the homepage index.html.
- The /generate route handles form submission, processes user input, generates the comic panels and returns the comic preview page.
- The /generate-comic/json route accepts a JSON payload, triggers comic generation and returns comic layout data and the generated PDF path.
- The /export-success route displays a confirmation page after the comic is downloaded.
- The /test-image route provides a developer utility for testing image generation independently.
- The route handlers use FastAPI request/form mechanisms, Pydantic validation, HTTPException handling and try-except blocks for error handling.

PHASE 4 — MILESTONE 4: FRONTEND DEVELOPMENT

Activity 4.1 — Designing and Developing the User Interface

- `index.html` is the main entry point and collects Story Prompt, Main Character Name, Setting, Story Tone and Art Style.
- The interface uses organized form elements, simple typography, a scenic background, centered content, consistent buttons, padding, shadows and rounded panels.
- `comic_preview.html` displays the generated comic panel by panel, including panel title, AI-generated image, scene description, caption and narration.
- `export_success.html` displays confirmation after PDF export and provides a way to return to the homepage.

Activity 4.2 — Creating Dynamic Templates with FastAPI's Jinja2

- FastAPI's Jinja2Templates is used for dynamic HTML rendering.
- The `/` route renders `index.html`, `/generate` renders `comic_preview.html`, and `/export-success` renders `export_success.html`.
- Form field name attributes map to FastAPI `Form(...)` parameters. The comic preview uses a Jinja2 loop to display the generated layout dynamically.
- This integration connects frontend inputs, backend processing and AI-generated comic results into a continuous user flow.

PHASE 5 — MILESTONE 5: DEPLOYMENT

Activity 5.1 — Preparing the Application for Local Deployment

- A virtual environment is created to isolate project dependencies. The requirements.txt libraries are installed inside the environment.
- Sensitive values such as the Gemini API key and Hugging Face API token are configured as environment variables or stored in a .env file loaded with python-dotenv.
- The application is started with `uvicorn app.main:app --reload`.

Activity 5.2 — Testing and Verifying Local Deployment

- The document describes functional and integration testing of the complete workflow, including AI text generation, image rendering, layout construction and PDF export.
- The local application is accessed at `http://127.0.0.1:8000`.
- FastAPI interactive API documentation is available at `http://127.0.0.1:8000/docs`.
- The testing stage verifies that users can enter story details and generate a personalized comic through the complete end-to-end pipeline.

END-TO-END PROJECT FLOW

The supplied document describes the overall ComicCraft pipeline as: user enters a story idea and preferences → FastAPI receives the input → Gemini Flash creates the structured five-panel outline → Gemini Pro creates narration and dialogue → Stable Diffusion generates panel illustrations → layout_builder.py assembles the panel data → exporters.py creates the PDF → the user receives the comic preview and export confirmation.

Key Project Components

Component	Role
Gemini Flash	Structured five-panel comic outline generation
Gemini Pro	Detailed narration and character dialogue generation
Stable Diffusion	Comic-style illustration generation
FastAPI	Backend routing, input processing and workflow coordination
Jinja2	Dynamic rendering of frontend templates
layout_builder.py	Combines images and story content into a structured layout
exporters.py / FPDF	Creates the final multi-page comic PDF

Source basis: ComicCraft - AI Comic Story Creator using Gemini Models.pdf supplied in this conversation. This phase-wise document preserves the milestone/activity structure and terminology from that source.